

Applied Machine Learning  
Unit 01 – Lecture 01 Notes  
Introduction to ML & the Python ML Stack

Tofik Ali

Autumn 2026

## Contents

|           |                                                  |          |
|-----------|--------------------------------------------------|----------|
| <b>1</b>  | <b>Lecture Overview</b>                          | <b>2</b> |
| <b>2</b>  | <b>How to use these notes (self-study)</b>       | <b>2</b> |
| <b>3</b>  | <b>Learning Outcomes</b>                         | <b>2</b> |
| <b>4</b>  | <b>Keywords (quick definitions)</b>              | <b>2</b> |
| <b>5</b>  | <b>Course roadmap (how topics connect)</b>       | <b>3</b> |
| <b>6</b>  | <b>Assessment (summary)</b>                      | <b>3</b> |
| <b>7</b>  | <b>Abbreviations used in this lecture</b>        | <b>4</b> |
| <b>8</b>  | <b>What machine learning is</b>                  | <b>4</b> |
| 8.1       | The shape of the problem . . . . .               | 4        |
| 8.2       | Two ways to build a spam filter . . . . .        | 4        |
| 8.3       | A working definition (Mitchell, 1997) . . . . .  | 5        |
| 8.4       | Where ML sits . . . . .                          | 5        |
| 8.5       | Why now . . . . .                                | 5        |
| 8.6       | Applications . . . . .                           | 5        |
| 8.7       | When not to use machine learning . . . . .       | 6        |
| <b>9</b>  | <b>Types of machine learning</b>                 | <b>6</b> |
| 9.1       | Supervised learning . . . . .                    | 6        |
| 9.2       | Unsupervised learning . . . . .                  | 6        |
| 9.3       | Reinforcement learning . . . . .                 | 7        |
| 9.4       | Naming the problem . . . . .                     | 7        |
| <b>10</b> | <b>The Python machine-learning stack</b>         | <b>7</b> |
| 10.1      | NumPy — vectorisation . . . . .                  | 7        |
| 10.2      | Pandas — the DataFrame . . . . .                 | 7        |
| 10.3      | scikit-learn — one API for every model . . . . . | 8        |
| 10.4      | End to end in twelve lines . . . . .             | 8        |
| <b>11</b> | <b>Practice questions</b>                        | <b>9</b> |

|                            |    |
|----------------------------|----|
| 12 Answers                 | 9  |
| 13 Before the next session | 9  |
| 14 Summary                 | 10 |

## 1 Lecture Overview

This lecture opens the course and answers four practical questions:

- What do we mean by **machine learning**, and when is it the right tool?
- How do we say precisely that a program has **learned** something?
- What are the **types** of machine learning, and how do we tell which one a problem needs?
- What **tools** will we use, and what does a complete model look like end to end?

Mapped to **CO1** (core concepts and techniques of ML and AI) and **CO2** (develop models using popular libraries and frameworks).

*Sources:* Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

## 2 How to use these notes (self-study)

Read in order and pause at each example to ask: “What would I do next, and why?”. In applied machine learning the important skill is not memorising definitions but connecting **problem type** → **method** → **evaluation**. Attempt the practice questions in Section 11 before reading the answers.

## 3 Learning Outcomes

- Explain what machine learning is and when it is the right tool.
- State Mitchell’s definition in terms of task, performance measure and experience.
- Distinguish supervised, unsupervised and reinforcement learning, and name the task type for a given problem.
- Describe the roles of NumPy, Pandas and scikit-learn, and run a complete model in a dozen lines.

## 4 Keywords (quick definitions)

- **Machine Learning (ML):** methods that learn a rule from examples in order to make predictions or decisions.
- **Model:** a simplified representation of a process, fitted to data and used for prediction.
- **Feature:** an input variable, one column of  $x$ .
- **Label / target:** the known answer  $y$  in supervised learning.

- **Training set / test set:** data used to fit the model, and data held back to estimate how it will behave on unseen cases.
- **Generalisation:** performance on unseen data drawn from the same process as the training data.
- **Overfitting:** fitting noise in the training data, so that training performance is good but test performance is poor.
- **Data leakage:** unintended use of information that would not be available at prediction time.
- **Loss function:** a number measuring how wrong a prediction is (Unit II).

## 5 Course roadmap (how topics connect)

The subject is organised from **what we measure** to **how we optimise** to **the methods themselves**:

- **Unit I (this unit):** what ML is, its types, and the Python stack.
- **Unit II:** loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adadelata, Adam.
- **Unit IV:** data preprocessing — cleaning, features, splitting, cross-validation, imbalance.
- **Unit V:** regression — linear, logistic, multiple, regularization.
- **Unit VI:** classification — KNN, trees, Bayes, ensembles, neural networks, SVM, evaluation.
- **Unit VII:** clustering — hierarchical, K-means, K-medoids, CLARA, DBSCAN.

Units II and III come early deliberately. Once you know **what quantity we are minimising** and **how the minimisation is carried out**, every method in Units V–VII is a variation on one theme.

## 6 Assessment (summary)

| Component                                            | Marks      |
|------------------------------------------------------|------------|
| Theory assignments (2, each with an assessment test) | 5          |
| Theory quizzes (10 short tests)                      | 10         |
| Project — individual quiz                            | 10         |
| Project — group submission state                     | 5          |
| Lab assignments (10, over experiments 1–14)          | 10         |
| Lab quizzes (10, code understanding)                 | 10         |
| Mid-Semester examination                             | 20         |
| End-Semester examination                             | 30         |
| <b>Total</b>                                         | <b>100</b> |

The project carries 15 marks, of which **10 are individual**. A strong contributor in a weak group can still score well; a passenger cannot.

**Test-0** is a 30-minute diagnostic on prerequisites and general understanding, with the date to be announced. It **carries no marks**. Its purpose is to let the teaching team see how you approach a problem, so that the short tests through the semester can be framed to suit how you work. There is nothing to revise; show your working, and if you do not know something, write what you would need to find out.

## 7 Abbreviations used in this lecture

- **ML**: Machine Learning    **AI**: Artificial Intelligence    **DL**: Deep Learning
- **EDA**: Exploratory Data Analysis (lab focus)    **MSE**: Mean Squared Error (Unit II)
- **KNN**: K-Nearest Neighbours (Unit VI)

## 8 What machine learning is

### 8.1 The shape of the problem

Some programs we can write by hand because we know the rule: sorting a list, parsing a date, computing a payroll. We state the rule and encode it.

Other programs we cannot write by hand. Decide whether an email is spam. Read a handwritten digit. Tell whether a scan shows a tumour. We can *recognise* the right answer when we see it, but we cannot write down the rule that produces it.

**Machine learning is what you reach for when you have examples of the answer but not a statement of the rule.** The examples are used to find the rule. That is the *supervised* case, which is most of this course; the other two families relax it, as the next section shows.

### 8.2 Two ways to build a spam filter

A rule-based filter encodes conditions directly:

```
def classify(subject, sender, bangs):
    if "free money" in subject: return "SPAM"
    if sender in blocklist: return "SPAM"
    if bangs > 5: return "SPAM"
    return "HAM"
```

The spammer writes “fr33 m0ney”. You add a rule. They adapt. A learned filter instead consumes labelled examples:

```
model.fit(emails, labels)
model.predict(new_email)
```

When the spammer adapts, you collect newly labelled mail and refit. What changed is not only the code but the **maintenance strategy**.

The rule-based filter does keep one property the learned one loses: you can read exactly why it fired. That property is **interpretability**, and recovering some of it for learned models is part of Unit VI (decision trees, rule extraction).

### 8.3 A working definition (Mitchell, 1997)

*A computer program is said to learn from experience  $E$  with respect to some class of tasks  $T$  and performance measure  $P$ , if its performance at tasks in  $T$ , as measured by  $P$ , improves with experience  $E$ .*

|              | Task $T$       | Experience $E$     | Performance $P$               |
|--------------|----------------|--------------------|-------------------------------|
| Spam filter  | classify email | labelled inbox     | fraction correctly classified |
| House prices | predict price  | past sales records | mean squared error            |
| Game agent   | choose a move  | games played       | win rate                      |

The definition forces you to name  $P$  **before** you start. A great many failed machine-learning projects failed at this step, not at the modelling step: nobody had agreed what “working” would mean.

### 8.4 Where ML sits

Artificial intelligence is the broad field: search, planning, logic, knowledge representation, and learning. Machine learning is the part of AI that learns the rule from data. Deep learning is a subset of ML that uses multi-layer neural networks.

Deep learning is a subset, not a synonym and not an automatic upgrade. On tabular data of the size you will meet in this course, a gradient-boosted tree typically beats a neural network and trains in seconds. Choosing the smallest model that solves the problem is a skill, not a compromise.

### 8.5 Why now

Three things changed together:

- **Data.** Transactions, sensors, images and logs are digitised and cheap to store, so the examples exist.
- **Compute.** Parallel hardware made the arithmetic of Unit III affordable at scale.
- **Algorithms and tooling.** Better optimisers and regularisation, and libraries that put all of it three lines away.

Most of the core ideas in this course are old — least squares dates from about 1805. What changed is that data and compute caught up with the mathematics.

### 8.6 Applications

| Domain      | Task                                          | Unit       |
|-------------|-----------------------------------------------|------------|
| Finance     | credit scoring, fraud detection               | VI         |
| Healthcare  | diagnosis from imaging, signal classification | VI, VII    |
| Retail      | churn prediction, customer segmentation       | V, VI, VII |
| Agriculture | crop disease from leaf images                 | VI         |
| Web         | spam filtering, recommendation                | VI, VII    |
| Operations  | demand and price forecasting                  | V          |

Most rows above appear as a lab experiment in this course.

## 8.7 When not to use machine learning

1. **The rule is known and stable.** Income tax is a lookup, not a prediction.
2. **You have no labelled examples** and no realistic way to obtain them.
3. **Errors are unacceptable** and you cannot bound them.
4. **Every decision must be explained** to a regulator, and a simple rule would do nearly as well.
5. **Training data will not resemble deployment.** A model trained on last year's customers predicts last year's customers.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

## 9 Types of machine learning

The question that decides which family you are in: **do you already have the answers?**

| Family        | What you have                      | What you learn          |
|---------------|------------------------------------|-------------------------|
| Supervised    | labelled data $\{(x_i, y_i)\}$     | a mapping $x \mapsto y$ |
| Unsupervised  | unlabelled data $\{x_i\}$          | structure in $x$        |
| Reinforcement | an environment and a reward signal | a policy                |

### 9.1 Supervised learning

Given  $\{(x_i, y_i)\}_{i=1}^n$ , find  $f$  such that  $f(x) \approx y$  on data *not seen during training*.

- **Regression** —  $y$  is continuous. House price, stock value, temperature, time to failure. Unit V; lab experiments 3–4.
- **Classification** —  $y$  is a category. Spam or not, digit 0–9, benign or malignant. Unit VI; lab experiments 5–9 and 11–12.

Fitting the training data is easy — a model can simply memorise it. The task is to be right on data never seen before. That single requirement is why Unit IV (splitting, cross-validation) and the over-fitting material in Unit V exist.

### 9.2 Unsupervised learning

No  $y$  is available.

- **Clustering** — group similar records: customer segments, disease subtypes. Unit VII.
- **Dimensionality reduction** — fewer features carrying substantially the same information. Unit IV.
- **Anomaly detection** — identify what does not fit. Lab experiment 10.

Unsupervised results have no ground truth to check against, so evaluating a clustering is genuinely harder than evaluating a classifier. Unit VII therefore spends real time on cluster validity measures.

### 9.3 Reinforcement learning

An agent takes actions in an environment and receives reward. There are no labelled answers, only consequences, and those consequences are often delayed. Used in robotics, game playing, control and feedback-driven recommendation. Covered here for orientation only.

### 9.4 Naming the problem

| Situation                                                           | Family        | Task              |
|---------------------------------------------------------------------|---------------|-------------------|
| 10 000 past loans marked repaid or defaulted; score a new applicant | supervised    | classification    |
| 5 years of daily sales; forecast next month                         | supervised    | regression        |
| 50 000 customer records, no labels; find natural groups             | unsupervised  | clustering        |
| Card transactions; flag the unusual ones                            | unsupervised  | anomaly detection |
| A warehouse robot that must learn a route                           | reinforcement | policy learning   |

## 10 The Python machine-learning stack

Three libraries, three jobs, stacked on one another:

|                     |                                                    |
|---------------------|----------------------------------------------------|
| <b>scikit-learn</b> | models, preprocessing, evaluation, pipelines       |
| <b>Pandas</b>       | tabular data: load, clean, reshape, group          |
| <b>NumPy</b>        | the <code>ndarray</code> and fast array arithmetic |

A Pandas `DataFrame` holds NumPy arrays; a scikit-learn estimator consumes NumPy arrays. Understanding the bottom layer is what lets you debug the top one. Matplotlib and Seaborn join the stack for plotting from Unit IV onwards.

### 10.1 NumPy — vectorisation

```
# Python loop
total = 0.0
for i in range(len(y)):
    d = y[i] - yhat[i]
    total += d * d
mse = total / len(y)

# NumPy
import numpy as np
mse = np.mean((y - yhat) ** 2)
```

The NumPy version is shorter and typically 10–100 times faster, because the loop runs in compiled code over a contiguous block of memory.

You have just written the **Mean Squared Error** — the first loss function of Unit II. Every optimiser in Unit III exists to make that number smaller.

Essentials to be comfortable with before lab 1: `np.array`, `.shape`, `.reshape`, broadcasting, the `axis` argument, and boolean masks.

### 10.2 Pandas — the DataFrame

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
df.head() # look at it first, always
df.info() # dtypes and non-null counts
df.describe() # numeric summary
df.isna().sum() # where is the data missing?

df["price_per_sqft"] = df["price"] / df["area"]
df.groupby("city")["price"].mean()
```

Build the habit now: run `head`, `info`, `describe` and `isna().sum()` before modelling anything. Unit IV is the disciplined version of that reflex.

### 10.3 scikit-learn — one API for every model

```
est.fit(X, y) # learn from data
est.predict(X) # apply to new data
est.score(X, y) # evaluate

tr.fit(X) # transformers follow the same shape
tr.transform(X)
```

Because the API is uniform, swapping a decision tree for a support vector machine changes one line. Learn it once here and every model in Units V–VII is already familiar.

**The rule you must not break.** Call `fit` on training data only — never on the test set, not even for scaling. Fitting a scaler on the full dataset before splitting leaks information about the test set into training and inflates your reported score. This is the single most common bug in student ML code, and a favourite planted fault in lab quizzes.

### 10.4 End to end in twelve lines

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)

model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

Read what each argument protects you from. `stratify=y` preserves the class balance across the split. `random_state` makes the result reproducible. `make_pipeline` guarantees the scaler is fitted inside the training portion only — the leak-proof way to combine preprocessing with a model.

*Sources:* Müller and Guido, *Introduction to Machine Learning with Python*, companion notebooks.; McKinney, *Python for Data Analysis* (3e), O'Reilly, 2022.; Scikit-learn user guide, accessed 2026-08-04.; Matplotlib and Seaborn documentation, accessed 2026-08-04.

## 11 Practice questions

Attempt these before reading the answers in Section 12.

1. State Mitchell's  $T$ ,  $P$  and  $E$  for a system that recommends films to a streaming subscriber.
2. A hospital wants to predict which patients will miss an appointment. Which of the five cautions in Section 8.7 is most likely to bite, and why?
3. Classify each as supervised regression, supervised classification, unsupervised or reinforcement: (a) predicting tomorrow's electricity demand; (b) grouping news articles by topic with no labels; (c) deciding whether a transaction is fraudulent given past confirmed fraud; (d) tuning a thermostat that is rewarded for comfort and penalised for energy use.
4. Explain in your own words why fitting a scaler before the train/test split inflates the reported score.
5. Rewrite this loop using NumPy: `s = 0 ; for i in range(len(a)): s += abs(a[i] - b[i]) ; mae = s / len(a)`.

## 12 Answers

1.  $T$ : recommend a film for a given subscriber.  $E$ : past viewing and ratings history.  $P$ : a stated measure such as click-through rate or watch-time on recommended titles. Any defensible  $P$  is acceptable provided it is named explicitly and could actually be measured.
2. Most likely caution 5, **distribution shift**: appointment behaviour depends on clinic policy, transport and season, all of which change, so a model trained on old data degrades. Caution 4 (explainability) is also defensible if the prediction is used to allocate scarce slots.
3. (a) supervised regression; (b) unsupervised clustering; (c) supervised classification; (d) reinforcement learning.
4. The scaler's mean and standard deviation are computed using test rows as well as training rows. The model is therefore evaluated on data whose statistics it has already seen, so the test score is no longer an honest estimate of performance on genuinely unseen data. Use `make_pipeline` so the scaler is fitted inside the training portion only.
5. `mae = np.mean(np.abs(a - b))`. This is the Mean Absolute Error, which we meet again in Unit II.

## 13 Before the next session

1. Install Python 3.11 or later (Anaconda or a `venv`).
2. `pip install numpy pandas scikit-learn matplotlib seaborn jupyter`
3. Run the twelve-line Iris script in a Jupyter notebook and print the version of each library.
4. Attempt the practice questions above.

Lab assignment LA1 covers experiments 1–2 (environment setup, then preprocessing and exploratory data analysis); it is due after lab session 2 and is examined by quiz LQ1 in lab session 3. Lab quizzes ask you to read, trace, debug and modify *your own* submitted code.

## 14 Summary

- Machine learning suits problems where you have examples of the answer but cannot state the rule.
- Mitchell’s  $T$ ,  $P$ ,  $E$  forces you to name the performance measure before modelling.
- Supervised (regression, classification), unsupervised (clustering, dimensionality reduction, anomaly detection), reinforcement (policy from reward).
- The goal is never to fit the training data — it is to generalise.
- NumPy → Pandas → scikit-learn, with one `fit`/`predict` API for every model.
- Never `fit` on the test set.

**Next (L2):** loss functions — putting a number on how wrong a prediction is. MSE, MAE and Huber for regression; cross-entropy for classification; hinge and triplet for margins.

## Reading

- Müller & Guido, *Introduction to Machine Learning with Python*, Ch. 1.
- Bishop, *Pattern Recognition and Machine Learning*, Ch. 1, §1.1.
- Murphy, *Probabilistic Machine Learning: An Introduction*, Ch. 1 (open access).
- James et al., *An Introduction to Statistical Learning with Python*, Ch. 1–2 (open access).
- Scikit-learn user guide, “Getting Started”.

Free, author-hosted copies: *An Introduction to Statistical Learning with Python* at <https://www.statlearning.com/>, Murphy’s *Probabilistic Machine Learning* at <https://probml.github.io/pml-book/>, and Bishop’s *Pattern Recognition and Machine Learning* from Microsoft Research.

*Sources:* Müller and Guido, *Introduction to Machine Learning with Python*, O’Reilly, 2016.; Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.; Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.; Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Morgan Kaufmann, 2012.